

Practical Week-6

Task-1:

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the Server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously

```
GNU nano 7.2 p6.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main() {
    char message[100];
    int fd_read, fd_write;

    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    printf("Server started. Waiting for client...\n");

    fd_read = open(FIFO1, O_RDONLY);
    read(fd_read, message, sizeof(message));
    close(fd_read);

    printf("Server received: %s\n", message);

    fd_write = open(FIFO2, O_WRONLY);

    char response[] = "Message received by server.";
    write(fd_write, response, strlen(response) + 1);

    close(fd_write);

    printf("Server response sent.\n");

    unlink(FIFO1);
    unlink(FIFO2);
}
```

main terminal:

```
root@HP:~# nano p6.c
root@HP:~# gcc p6.c -o server
root@HP:~# ./server
Server started. Waiting for client...
|
```

Terminal 2:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <fcntl.h>
5  #include <unistd.h>
6
7  #define FIFO1 "client_to_server"
8  #define FIFO2 "server_to_client"
9
10 int main() {
11     char message[100];
12     char response[100];
13     int fd_write, fd_read;
14
15     printf("Enter message: ");
16     fgets(message, sizeof(message), stdin);
17
18     message[strcspn(message, "\n")] = '\0';
19
20     fd_write = open(FIFO1, O_WRONLY);
21
22     write(fd_write, message, strlen(message) + 1);
23     close(fd_write);
24
25     fd_read = open(FIFO2, O_RDONLY);
26
27     read(fd_read, response, sizeof(response));
28     close(fd_read);
29
30     printf("Server response: %s\n", response);
31
32     return 0;
33 }
```

Main terminal:

```
root@HP: ~
root@HP:~# nano client.c
root@HP:~# gcc client.c -o client
root@HP:~# ./client
Enter message: hello server
Server response: Message received by server.
root@HP:~#
```

Terminal 2:

```
root@HP: ~
root@HP:~# nano p6.c
root@HP:~# gcc p6.c -o server
root@HP:~# ./server
Server started. Waiting for client...
Server received: hello server
Server response sent.
root@HP:~#
```

Task-2:

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
GNU nano 7.2 p62.c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int signal) {
    if (signal == SIGINT) {
        printf("\nSIGINT received (Ctrl+C).\n");
    }
    else if (signal == SIGTERM) {
        printf("SIGTERM received.\n");
    }
    else if (signal == SIGUSR1) {
        printf("SIGUSR1 received.\n");
    }

    fflush(stdout);
}

int main() {
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Signal handling program started.\n");
    printf("PID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1) {
        pause();
    }

    return 0;
}
```

[Wrote 34 lines]

```
root@HP:~# nano p62.c
root@HP:~# gcc p62.c -o signal
root@HP:~# ./signal
Signal handling program started.
PID: 556
Waiting for signals...
|
```

```
root@HP: ~
root@HP:~# nano p62.c
root@HP:~# gcc p62.c -o signal
root@HP:~# ./signal
Signal handling program started.
PID: 556
Waiting for signals...
SIGUSR1 received.

root@HP:~# kill -SIGUSR1 556
root@HP:~#

root@HP:~# nano p62.c
root@HP:~# gcc p62.c -o signal
root@HP:~# ./signal
Signal handling program started.
PID: 556
Waiting for signals...
SIGUSR1 received.
SIGTERM received.

root@HP:~# kill -SIGUSR1 556
root@HP:~# kill -SIGTERM 1234
-bash: kill: (1234) - No such process
root@HP:~# kill -SIGTERM 556
root@HP:~#
```

Terminal 1:

```
Waiting for signals...
SIGUSR1 received.
SIGTERM received.
^C
SIGINT received (Ctrl+C).
```

Terminal 1 & 2:

```
waiting for signals...
SIGUSR1 received.
SIGTERM received.
^C
SIGINT received (Ctrl+C).
Killed
root@HP:~# |
```

```
root@HP: ~
root@HP:~# kill -SIGUSR1 556
root@HP:~# kill -SIGTERM 1234
-bash: kill: (1234) - No such process
root@HP:~# kill -SIGTERM 556
root@HP:~# ^C
root@HP:~# kill -9 556
root@HP:~#
```

Killed.